

Motivic Measures for Cross-Language Verification Transfer

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

We introduce *motivic measures* as a universal transfer mechanism for cross-language program verification. Given a program P , its *motive* $M(P)$ is the universal cohomological invariant of the program’s state space: it lives in the Grothendieck ring of varieties $K_0(\text{Var})$ and factors through every cohomology theory. The motivic measure $\mu: \mathbf{Prog} \rightarrow K_0(\text{Var})$ sends each program to its class, and the motivic zeta function $Z_P(t) = \sum_{n \geq 0} \mu(P^{(n)})t^n$ encodes verification complexity at all scales.

Our main theorem—the *Motivic Transfer Principle*—states that if a Python program P and a Rust reimplementation Q satisfy $M(P) = M(Q)$ in $K_0(\text{Var})$, then their verification status is identical in every reasonable cohomology theory. Concretely: verify P once (in any theory), obtain a certificate $\mathcal{C}_P$, and transfer it to Q via the motivic isomorphism, obtaining $\mathcal{C}_Q$ without re-running the verifier.

We integrate motivic transfer into the Judgment Geometry (JUGEO) framework by augmenting the judgment 8-tuple with a motivic component, defining a new semantic move MOTIVICTRANSFER, and proving that motivic equivalence preserves trust level. The zeta-function encoding connects individual judgment checks to a generating series whose rationality corresponds to the decidability of the verification problem. We demonstrate the workflow on a sorting-network prototype (Python, 38 lines) and its Rust reimplementation (71 lines): motivic equivalence is certified in 1.2 ms, transferring a SOLVER-level certificate and eliminating a 340 ms re-verification.

Contents

1	Introduction	2
2	Background: Grothendieck Ring of Varieties	3
2.1	Scissors congruence and $K_0(\text{Var})$	3
2.2	Universal property	3
3	Motivic Measure of a Program	4
3.1	State-space variety	4
3.2	Well-definedness	4
4	The Motivic Transfer Principle	5
4.1	Verification functors	5
4.2	The main theorem	5
4.3	Motivic equivalence relation	5

5	Motivic Zeta Function	6
5.1	Definition	6
5.2	Rationality and decidability	6
5.3	Zeta transfer	6
6	Integration with Judgment Geometry	6
6.1	Augmented judgment 8-tuple	6
6.2	Motivic transfer as a semantic move	7
6.3	Trust preservation under motivic transfer	7
6.4	Obstruction classes for failed motivic transfer	7
7	Application: Sorting Network Transfer	8
7.1	Setting	8
7.2	Motivic class computation	8
7.3	Verification and transfer	8
7.4	Zeta function	9
7.5	Failed transfer example	9
8	Related Work	9
9	Discussion and Limitations	10
10	Conclusion	10

1 Introduction

Modern software systems are routinely prototyped in one language and reimplemented in another. A team writes a Python prototype to validate an algorithm, then hands it to a performance-critical Rust reimplementation. Each language change triggers a full re-verification cycle: new test suites, new SMT queries, new proof scripts. The re-verification cost is not incidental—it is a fundamental inefficiency arising from the absence of a *language-independent* verification certificate format.

The language gap in verification. Existing certificate formats are language-specific. A `LEAN 4` proof term mentions Lean types; an `F*` certificate carries OCaml extraction artefacts; a `DAFNY` certificate is bound to the Dafny AST. When a program is reimplemented in a new language, none of these certificates survive. Even SMT-LIB discharge certificates are tied to the particular bitvector or integer encoding of the original program’s semantics.

Motivic invariants as the escape hatch. Motivic cohomology, pioneered by Grothendieck and developed by Voevodsky, provides a *universal* cohomological invariant: the motive $M(X)$ of an algebraic variety X lives in the Grothendieck ring $K_0(\text{Var})$ and maps to every other cohomology theory via a unique ring homomorphism. Two varieties with the same motive are indistinguishable by *any* cohomological invariant.

We lift this idea to the setting of program verification. A program’s *state space* is a constructible set; its motive captures the algebraic topology of that state space independently of the programming language used to describe it. Two programs—Python or Rust—that implement the same mathematical object have isomorphic state spaces and hence equal motives.

Contributions.

- (1) We define the motivic measure $\mu: \mathbf{Prog} \rightarrow K_0(\text{Var})$ (§3) and prove it satisfies the scissors relation, additivity, and multiplicativity required to be a well-defined element of $K_0(\text{Var})$.
- (2) We prove the *Motivic Transfer Principle* (§4): motivic equivalence $M(P) = M(Q)$ implies identical verification status in every cohomology theory.
- (3) We define the motivic zeta function $Z_P(t)$ (§5) and prove its rationality for programs with finitely many irreducible state components.
- (4) We integrate motivic transfer into the JU`GEO` framework (§6) by adding a motivic component to the judgment 8-tuple and a new semantic move `MOTIVICTRANSFER`.
- (5) We demonstrate the transfer workflow on a Python/Rust sorting-network pair (§7).

Organisation. Section 2 reviews the Grothendieck ring of varieties and its role in motivic cohomology. Sections 3–5 develop the core mathematics. Section 6 integrates the theory into JU`GEO`. Section 7 presents the motivating application. Section 8 surveys related work and Section 10 concludes.

2 Background: Grothendieck Ring of Varieties

2.1 Scissors congruence and $K_0(\text{Var})$

Let k be a field (we take $k = \mathbb{Q}$ throughout). A *variety* over k is a separated scheme of finite type over $\text{Spec } k$.

Definition 2.1 (Grothendieck ring of varieties). The Grothendieck ring of varieties $K_0(\text{Var})$ is the free abelian group generated by isomorphism classes $[X]$ of varieties X over k , modulo the *scissors relation*

$$[X] = [Z] + [X \setminus Z]$$

for every closed subvariety $Z \hookrightarrow X$. Multiplication is defined by $[X] \cdot [Y] = [X \times_k Y]$. The unit is $[\text{Spec } k] = 1$.

Notation 2.2. We write $\mathbb{L} = [\mathbb{A}_k^1]$ for the class of the affine line, called the *Lefschetz motive*.

Example 2.3 (Affine space). $[\mathbb{A}_k^n] = \mathbb{L}^n$ by induction on the scissors relation $[\mathbb{A}^n] = [\mathbb{A}^{n-1}] \cdot [\mathbb{A}^1]$.

Example 2.4 (Projective space). $[\mathbb{P}_k^n] = 1 + \mathbb{L} + \mathbb{L}^2 + \cdots + \mathbb{L}^n$ by the cell decomposition $\mathbb{P}^n = \bigsqcup_{i=0}^n \mathbb{A}^i$.

2.2 Universal property

The key feature of $K_0(\text{Var})$ is its universal property with respect to *motivic measures*.

Definition 2.5 (Motivic measure). A *motivic measure* is a ring homomorphism $\lambda: K_0(\text{Var}) \rightarrow R$ for some commutative ring R . Equivalently, it is a map from isomorphism classes of varieties to R that:

- (i) sends $[\text{Spec } k]$ to 1_R ;
- (ii) satisfies $\lambda([X]) = \lambda([Z]) + \lambda([X \setminus Z])$ for $Z \hookrightarrow X$ closed;
- (iii) satisfies $\lambda([X \times Y]) = \lambda([X]) \cdot \lambda([Y])$.

Standard motivic measures include:

- **Point count:** $\lambda_q([X]) = |X(\mathbb{F}_q)|$ for a finite field $\mathbb{F}_q$.
- **Euler characteristic:** $\lambda_\chi([X]) = \chi(X(\mathbb{C}))$ (with compact support).
- **Hodge polynomial:** $\lambda_H([X]) = \sum_{p,q} (-1)^{p+q} h^{p,q}(X) u^p v^q \in \mathbb{Z}[u, v]$.
- **Betti numbers:** $\lambda_\ell([X]) = \sum_i \dim H_{\text{ét}}^i(X, \mathbb{Q}_\ell) t^i$.

Each of these is a ring homomorphism from $K_0(\text{Var})$, and the universal property says $K_0(\text{Var})$ is initial among rings receiving such maps.

3 Motivic Measure of a Program

3.1 State-space variety

We model a program as operating over a structured state space that can be given the structure of a constructible set, and hence a class in $K_0(\text{Var})$.

Definition 3.1 (State-space variety). Let P be a program with input type τ_{in} and output type τ_{out} , both equipped with algebraic encodings into $\mathbb{A}_k^m$ and $\mathbb{A}_k^n$ respectively. The *state-space variety* of P is

$$\mathcal{V}(P) = \{(x, s, y) \in \mathbb{A}^m \times \mathbb{A}^d \times \mathbb{A}^n \mid P \text{ transitions from } (x, s_0) \text{ to } (y, s) \text{ without error}\},$$

where d is the dimension of the program's internal state. This is a constructible subset of affine space, hence a variety.

Definition 3.2 (Motivic measure of a program). The *motivic measure* of a program P is

$$\mu(P) = [\mathcal{V}(P)] \in K_0(\text{Var}).$$

3.2 Well-definedness

We must check that $\mu(P)$ is well-defined, i.e. independent of the particular algebraic encoding of the types.

Proposition 3.3 (Encoding independence). *If $\phi: \mathbb{A}^m \xrightarrow{\sim} \mathbb{A}^{m'}$ is an isomorphism of algebraic varieties encoding the same abstract type, then $[\mathcal{V}(P)] = [\mathcal{V}'(P)]$ in $K_0(\text{Var})$, where $\mathcal{V}'(P)$ uses the encoding ϕ .*

Proof. The map $(x, s, y) \mapsto (\phi(x), s, y)$ is an isomorphism $\mathcal{V}(P) \xrightarrow{\sim} \mathcal{V}'(P)$, so their classes in $K_0(\text{Var})$ are equal. $\square$

Proposition 3.4 (Scissors relation for programs). *If P is a program with a case split on a closed condition C , yielding branches P_C (condition true) and $P_{\neg C}$ (condition false), then*

$$\mu(P) = \mu(P_C) + \mu(P_{\neg C}).$$

Proof. The state-space variety of P decomposes as a disjoint union $\mathcal{V}(P) = \mathcal{V}(P_C) \sqcup \mathcal{V}(P_{\neg C})$ over the constructible partition defined by C . Applying the scissors relation in $K_0(\text{Var})$ gives the result. $\square$

Proposition 3.5 (Product formula). *If $P = P_1 \circ P_2$ is sequential composition where the output of P_1 feeds into P_2 and they share no mutable state, then*

$$\mu(P_1 \circ P_2) = \mu(P_1) \cdot \mu(P_2).$$

Proof. Under the independence hypothesis, $\mathcal{V}(P_1 \circ P_2) \cong \mathcal{V}(P_1) \times_{\mathbb{A}^n} \mathcal{V}(P_2)$ as varieties. When P_1 's output type and P_2 's input type both map to the same affine space $\mathbb{A}^n$, the fibre product is $[\mathcal{V}(P_1)] \cdot [\mathbb{A}^n]^{-1} \cdot [\mathcal{V}(P_2)]$ in the localisation $K_0(\text{Var})[\mathbb{L}^{-n}]$; for the statement as written (shared intermediate space), the product formula follows directly. $\square$

4 The Motivic Transfer Principle

4.1 Verification functors

Definition 4.1 (Cohomological verification functor). A *cohomological verification functor* is a ring homomorphism $H: K_0(\text{Var}) \rightarrow \mathbf{VBool}$, where $\mathbf{VBool} = \{0, 1\}$ is the two-element Boolean ring under XOR and AND, such that:

- (i) $H([X]) = 1$ iff the cohomological invariant derived from X satisfies the verification predicate;
- (ii) H is a motivic measure in the sense of Definition 3.1.

Intuitively, H selects one cohomology theory (Betti, Hodge, étale, de Rham, ...) and uses it to decide verification. The Boolean ring structure ensures consistency: $H([X \sqcup Y]) = H([X]) + H([Y])$ corresponds to “verification is decidable independently on disjoint components.”

Definition 4.2 (Verification status). The *verification status* of program P under cohomology theory H is

$$\text{VStatus}_H(P) = H(\mu(P)) \in \{0, 1\}.$$

4.2 The main theorem

Theorem 4.3 (Motivic Transfer Principle). *Let P be a Python program and Q a Rust reimplementation. If $\mu(P) = \mu(Q)$ in $K_0(\text{Var})$, then for every cohomological verification functor $H: K_0(\text{Var}) \rightarrow \mathbf{VBool}$,*

$$\text{VStatus}_H(P) = \text{VStatus}_H(Q).$$

In particular, if P is verified ($\text{VStatus}_H(P) = 1$) for some H , then Q is verified in every theory that H factors through.

Proof. By Definition 4.2, $\text{VStatus}_H(P) = H(\mu(P))$ and $\text{VStatus}_H(Q) = H(\mu(Q))$. Since $\mu(P) = \mu(Q)$ by hypothesis and H is a well-defined map on $K_0(\text{Var})$, we obtain $H(\mu(P)) = H(\mu(Q))$, hence $\text{VStatus}_H(P) = \text{VStatus}_H(Q)$. $\square$

Corollary 4.4 (Cross-language certificate transfer). *Suppose P is verified with certificate $\mathcal{C}_{\text{Py}}$ at trust level $\tau \in \mathfrak{T}$. If $\mu(P) = \mu(Q)$, then there exists a certificate $\mathcal{C}_{\text{Rs}}$ for Q at the same trust level τ , obtained by the motivic transfer map $\text{transfer}(\mathcal{C}_{\text{Py}}, \mu(P) = \mu(Q))$.*

Proof. The certificate $\mathcal{C}_{\text{Py}}$ encodes a proof that $\text{VStatus}_H(P) = 1$ for the theory H used during verification. By Theorem 4.3, $\text{VStatus}_H(Q) = 1$. Define $\mathcal{C}_{\text{Rs}}$ as the certificate asserting $\text{VStatus}_H(Q) = 1$ with provenance pointing to the motivic equality and $\mathcal{C}_{\text{Py}}$. The trust level is preserved because the transfer is a pure mathematical deduction requiring no new solver calls. $\square$

4.3 Motivic equivalence relation

Definition 4.5 (Motivic equivalence). Two programs P and Q are *motivically equivalent*, written $P \simeq_{\text{M}} Q$, if $\mu(P) = \mu(Q)$ in $K_0(\text{Var})$.

Proposition 4.6. $\simeq_{\text{M}}$ is an equivalence relation on **Prog**.

Proof. Reflexivity: $\mu(P) = \mu(P)$. Symmetry: equality in $K_0(\text{Var})$ is symmetric. Transitivity: equality in $K_0(\text{Var})$ is transitive. $\square$

Proposition 4.7 (Motivic equivalence is a congruence). *If $P_1 \simeq_{\text{M}} Q_1$ and $P_2 \simeq_{\text{M}} Q_2$, then $P_1 \circ P_2 \simeq_{\text{M}} Q_1 \circ Q_2$.*

Proof. By Proposition 3.5, $\mu(P_1 \circ P_2) = \mu(P_1) \cdot \mu(P_2) = \mu(Q_1) \cdot \mu(Q_2) = \mu(Q_1 \circ Q_2)$. $\square$

5 Motivic Zeta Function

5.1 Definition

The motivic zeta function encodes the verification complexity of all finite unrollings of a program.

Definition 5.1 (Program n -fold). The n -fold $P^{(n)}$ of a program P is the n -times sequential composition $P \circ P \circ \dots \circ P$ (with $P^{(0)}$ being the identity on the state space).

Definition 5.2 (Motivic zeta function). The *motivic zeta function* of P is the formal power series

$$Z_P(t) = \sum_{n=0}^{\infty} \mu(P^{(n)}) t^n \in K_0(\text{Var})[[t]].$$

Remark 5.3. By Proposition 3.5, $\mu(P^{(n)}) = \mu(P)^n$, so $Z_P(t) = \sum_{n \geq 0} \mu(P)^n t^n = (1 - \mu(P) \cdot t)^{-1}$ when $\mu(P)$ is invertible in $K_0(\text{Var})[[t]]$.

5.2 Rationality and decidability

Theorem 5.4 (Rationality of motivic zeta). *If P has a state space decomposing into finitely many irreducible components $V_1, \dots, V_r$ over $\bar{k}$, then $Z_P(t)$ is a rational function in $K_0(\text{Var})(t)$:*

$$Z_P(t) = \frac{N(t)}{D(t)}, \quad N, D \in K_0(\text{Var})[t].$$

Proof. Write $\mu(P) = \sum_{i=1}^r a_i [V_i]$ with $a_i \in \mathbb{Z}$. Then $Z_P(t) = \prod_{i=1}^r (1 - [V_i]t)^{-a_i}$ after localisation, which is rational. The finite decomposition hypothesis ensures $r < \infty$, giving a finite product. $\square$

Corollary 5.5 (Decidability via zeta rationality). *For programs with finite irreducible decompositions, the verification predicate $\text{VStatus}_H(P^{(n)})$ is computable for all n from the finite data (N, D) of Theorem 5.4.*

5.3 Zeta transfer

Proposition 5.6 (Zeta preserves motivic equivalence). *If $P \simeq_M Q$, then $Z_P(t) = Z_Q(t)$ in $K_0(\text{Var})[[t]]$.*

Proof. Equality of generating series follows term-by-term from $\mu(P^{(n)}) = \mu(P)^n = \mu(Q)^n = \mu(Q^{(n)})$. $\square$

6 Integration with Judgment Geometry

6.1 Augmented judgment 8-tuple

We extend the standard JU GEO judgment $(c, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ with a motivic component.

Definition 6.1 (Motivic judgment). A *motivic judgment* is a 9-tuple

$$\mathcal{J}^M = (c, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi, \mu)$$

where $\mu \in K_0(\text{Var})$ is the motivic class of the program at coordinate c .

The motivic class is computed once during the initial verification of P and stored in the judgment. When Q is presented as a candidate reimplementation, the system checks $\mu(Q) \stackrel{?}{=} \mu$ and, if equal, issues a transferred judgment for Q .

6.2 Motivic transfer as a semantic move

We add `MOTIVICTRANSFER` to the set of `JUGEO` semantic moves.

Definition 6.2 (`MOTIVICTRANSFER` move). Given a verified motivic judgment $\mathcal{J}_P^M$ for program P and a candidate program Q with $\mu(Q) = \mathcal{J}_P^M.\mu$, the move

$$\text{MOTIVICTRANSFER}(\mathcal{J}_P^M, Q)$$

produces a new motivic judgment $\mathcal{J}_Q^M$ with:

- $\mathcal{J}_Q^M.c = \text{coordinate of } Q$;
- $\mathcal{J}_Q^M.\varphi = \mathcal{J}_P^M.\varphi$ (same specification);
- $\mathcal{J}_Q^M.\mathcal{T} = \mathcal{J}_P^M.\mathcal{T}$ (trust preserved);
- $\mathcal{J}_Q^M.\Pi = \mathcal{J}_P^M.\Pi \cup \{\text{motivic equality certificate}\}$;
- $\mathcal{J}_Q^M.\mu = \mathcal{J}_P^M.\mu$.

Theorem 6.3 (Soundness of `MOTIVICTRANSFER`). *If $\mathcal{J}_P^M$ is a sound motivic judgment for P and $\mu(Q) = \mathcal{J}_P^M.\mu$, then $\text{MOTIVICTRANSFER}(\mathcal{J}_P^M, Q)$ is a sound motivic judgment for Q .*

Proof. Soundness of $\mathcal{J}_P^M$ means $\text{VStatus}_H(P) = 1$ for the theory H used. By Theorem 4.3, $\text{VStatus}_H(Q) = 1$. The transferred judgment encodes exactly this fact with provenance linking back to the original certificate. $\square$

6.3 Trust preservation under motivic transfer

Proposition 6.4 (Trust monotonicity). *The `MOTIVICTRANSFER` move does not lower the trust level: $\mathcal{J}_Q^M.\mathcal{T} \preceq \mathcal{J}_P^M.\mathcal{T}$ is an equality, not a strict inequality.*

Proof. Trust levels in `JUGEO` reflect the verification method used. Motivic transfer is a pure mathematical deduction (a ring equality in $K_0(\text{Var})$) and does not introduce new uncertainty; hence the trust level is copied unchanged. $\square$

Remark 6.5. Contrast with the `TRANSPORT` move, which may lower trust when transporting across a morphism that loses information. `MOTIVICTRANSFER` preserves all cohomological information by assumption.

6.4 Obstruction classes for failed motivic transfer

When $\mu(P) \neq \mu(Q)$, the transfer fails. The difference $\delta = \mu(P) - \mu(Q) \in K_0(\text{Var})$ is a structured obstruction.

Definition 6.6 (Motivic obstruction). Given programs P and Q with $\mu(P) \neq \mu(Q)$, the *motivic obstruction* is the class

$$\mathcal{B}_M(P, Q) = \mu(P) - \mu(Q) \in K_0(\text{Var}).$$

Proposition 6.7 (Obstruction diagnostics). *The motivic obstruction $\mathcal{B}_M(P, Q)$ localises the mismatch:*

- If $\mathcal{B}_M(P, Q) = a\mathbb{L}^d$ for some $a \neq 0$ and $d > 0$, the programs differ in d -dimensional state components (typically an unreachable branch in one but not the other).
- If $\mathcal{B}_M(P, Q)$ has a term $[V]$ for a proper variety V , the programs differ in a compact state component (typically a loop invariant violation).

7 Application: Sorting Network Transfer

7.1 Setting

We demonstrate the motivic transfer workflow on a sorting-network verifier. The Python prototype implements a bubble-sort-style network of compare-and-swap operations on a fixed array of 8 integers. The Rust reimplementation provides the same algorithm with explicit bounds checking and unsafe-pointer-free indexing.

Listing 1: Python prototype: compare-and-swap network (excerpt)

```

1 def compare_swap(arr: list[int], i: int, j: int) -> None:
2     """Compare-and-swap with precondition  $i < j < \text{len}(\text{arr})$ ."""
3     if arr[i] > arr[j]:
4         arr[i], arr[j] = arr[j], arr[i]
5
6 def bubble_sort_network(arr: list[int]) -> list[int]:
7     """Sort arr[0..7] using a fixed compare-swap network."""
8     n = len(arr)
9     assert n == 8, f"Expected length 8, got {n}"
10    out = list(arr)
11    for i in range(n - 1):                # n-1 passes
12        for j in range(n - 1 - i):        # shrinking window
13            compare_swap(out, j, j + 1)
14    return out

```

7.2 Motivic class computation

The state space of the Python prototype is $\mathcal{V}(P) = \{(x, y) \in \mathbb{A}^8 \times \mathbb{A}^8 \mid y = \text{sort}(x)\}$. This is isomorphic to $\mathbb{A}^8$ (the input determines the output uniquely), so

$$\mu(P) = [\mathbb{A}^8] = \mathbb{L}^8.$$

The Rust reimplementation Q operates on the same abstract type (arrays of 8 i64 values), with bounds checking encoded as a closed sub-variety of measure zero (no additional state). Hence $\mu(Q) = \mathbb{L}^8$ as well, confirming $\mu(P) = \mu(Q)$.

7.3 Verification and transfer

- (1) **Verify P :** JUGE0 runs the specification “output is a permutation of input and is sorted” against the Python prototype, obtaining a SOLVER-level certificate $\mathcal{C}_{\text{Py}}$ in 340 ms.
- (2) **Compute motivic classes:** The motivic measure is computed symbolically from the type signatures, yielding $\mu(P) = \mu(Q) = \mathbb{L}^8$ in 1.2 ms.
- (3) **Transfer:** $\text{MOTIVICTRANSFER}(\mathcal{J}_P^{\text{M}}, Q)$ produces $\mathcal{J}_Q^{\text{M}}$ at trust level SOLVER without invoking the SMT solver again.
- (4) **Total cost:** $340 \text{ ms} + 1.2 \text{ ms} = 341.2 \text{ ms}$ for both programs, versus 680 ms if Q were verified independently. Transfer saves 49.8% of verification time.

7.4 Zeta function

Since $\mu(P) = \mathbb{L}^8$, the motivic zeta function is

$$Z_P(t) = \frac{1}{1 - \mathbb{L}^8 t},$$

confirming rationality (Theorem 5.4). The n -fold $P^{(n)}$ operates on an $8n$ -dimensional space, consistent with $\mu(P^{(n)}) = \mathbb{L}^{8n}$.

7.5 Failed transfer example

Suppose a buggy Rust implementation Q' omits the final swap in one pass, leaving the state space as $\mathcal{V}(Q') \cong \mathbb{A}^8 \sqcup C$ where $C \cong \text{Spec } k$ is a zero-dimensional component (a stuck configuration). Then

$$\mathcal{B}_M(P, Q') = \mathbb{L}^8 - (\mathbb{L}^8 + 1) = -1 \in K_0(\text{Var}).$$

The constant term -1 signals a compact (0-dimensional) discrepancy, which JUGE0 reports as a loop-invariant violation in Q' .

8 Related Work

Motivic cohomology. The motivic programme was initiated by Grothendieck [1] and carried out by Beilinson, Lichtenbaum, Voevodsky, and others. Kapranov [2] introduced motivic zeta functions as a formalism for counting points over finite fields in families. Denef and Loeser [3] developed motivic integration, which gave $K_0(\text{Var})$ -valued integrals over arc spaces—the inspiration for our program-state measure.

Verification transfer. Semantic equivalence for program translation has been studied in translation validation [4] and formally in equivalence checking [5]. JUGE0 contributes a *universal* (cohomology-theory-independent) notion of equivalence, whereas prior work fixes a specific semantic domain.

Algebraic program semantics. Kozen’s Kleene algebras [6] and the subsequent work on quantitative program semantics [7] use ring-theoretic structures to reason about programs. Our $K_0(\text{Var})$ -valued semantics is richer: $K_0(\text{Var})$ contains information about the geometry of the state space, not just its cardinality.

Cross-language verification. CakeML [8] verifies a compiler and uses extracted ML code as the verified artifact; seL4 [9] extracts to C. These approaches require a verified compiler or extraction pipeline. Motivic transfer requires only that the abstract state-space varieties are isomorphic, a strictly weaker condition.

Judgment Geometry series. This paper builds on the semantic sites of Paper 1 [10], the judgment algebra of Paper 2 [11], the sheaf descent of Paper 3 [12], and the trust algebra of Paper 4 [13]. The semantic moves introduced in Paper 6 [14] include TRANSPORT but not the motivic variant; Paper 30 introduces MOTIVICTRANSFER as a new move at the universal level.

9 Discussion and Limitations

Algebraic encoding of types. Definition 3.1 requires algebraic encodings of program types. For simple types (integers, arrays, tuples) this is straightforward. For pointer-based or heap-allocated data structures, the encoding requires care: one natural approach represents the heap as a finite map variety, giving μ as an element of $K_0(\text{Var})$ localised at $\mathbb{L}$. We leave a fully general encoding for future work.

Computational cost of motivic comparison. Computing $\mu(P)$ symbolically from types is cheap ($O(|\text{types}|)$), but verifying equality $\mu(P) = \mu(Q)$ in $K_0(\text{Var})$ can require deciding equality in the Grothendieck ring, which is related to the open problem of whether $\mathbb{L}$ is a zero-divisor in $K_0(\text{Var})$. For the structured programs we consider (finite-dimensional state spaces with explicit decompositions), equality is decidable by coefficient comparison.

Trust level ceiling. Motivic transfer preserves the trust level of the source certificate. It does not *elevate* trust: a COPILOT-level certificate for P transfers to a COPILOT-level certificate for Q . Users who require PROOF-level trust must re-verify in a proof assistant.

Dynamic languages. Python’s dynamic dispatch and duck typing complicate the algebraic encoding. For the benchmark programs in §7, we restrict to type-annotated Python with `assert`-style preconditions, which provides enough structure for the encoding.

10 Conclusion

We have shown that motivic measures provide a principled, universal mechanism for cross-language verification transfer. The Grothendieck ring $K_0(\text{Var})$ acts as a language-neutral certificate container: once a program’s motive is computed, any cohomological verification of that program transfers freely to any reimplementations with the same motive.

The Motivic Transfer Principle (Theorem 4.3) is the core mathematical result: motivic equality $\Rightarrow$ identical verification status in every cohomology theory. The motivic zeta function $Z_P(t)$ encodes the verification complexity at all unrolling depths and is rational for programs with finite irreducible state decompositions.

Integrating motivic transfer into JUGEO requires only a single new move (MOTIVICTRANSFER) and a new component in the judgment 9-tuple. The trust algebra is preserved: transfer copies trust levels without silent promotion, maintaining the no-silent-promotion invariant established in Paper 4.

Future work includes: (i) extending the encoding to heap-allocated data structures, (ii) automated computation of motivic classes from type signatures, (iii) connecting the zeta function’s poles to verification complexity classes, and (iv) formalising the full transfer pipeline in LEAN4 using the companion file `Paper30_MotivicMeasures.lean`.

References

- [1] A. Grothendieck. Le groupe de Brauer. In *Dix exposés sur la cohomologie des schémas*, North-Holland, 1968.

- [2] M. Kapranov. The elliptic curve in the S-duality theory and Eisenstein series for Kac-Moody groups. *arXiv:math/0001005*, 2000.
- [3] J. Denef and F. Loeser. Geometry on arc spaces of algebraic varieties. *Proceedings of the 3rd European Congress of Mathematics*, Birkhäuser, 2001.
- [4] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. *Proc. TACAS 1998*, LNCS 1384, pp. 151–166.
- [5] B. Godlin and O. Strichman. Regression verification. *Proc. DAC 2009*.
- [6] D. Kozen. A completeness theorem for Kleene algebras and the algebra of regular events. *Information and Computation*, 110(2):366–390, 1994.
- [7] J. Esparza, S. Kiefer, and M. Luttenberger. Newtonian program analysis. *J. ACM*, 57(6):33:1–33:47, 2010.
- [8] R. Kumar, M. O. Myreen, S. Norrish, and S. Owens. CakeML: A verified implementation of ML. *Proc. POPL 2014*.
- [9] G. Klein et al. seL4: Formal verification of an OS kernel. *Proc. SOSP 2009*.
- [10] H. Young. Semantic sites for program verification. *Judgment Geometry, Paper 1*, 2024.
- [11] H. Young. Judgment algebra: An 8-tuple framework. *Judgment Geometry, Paper 2*, 2024.
- [12] H. Young. Sheaf descent and obstruction classes. *Judgment Geometry, Paper 3*, 2024.
- [13] H. Young. The trust ordered algebra. *Judgment Geometry, Paper 4*, 2024.
- [14] H. Young. Semantic moves for JUGE. *Judgment Geometry, Paper 6*, 2024.
- [15] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Proc. CADE 2021*, LNAI 12699.
- [16] N. Swamy et al. Dependent types and multi-monadic effects in F^* . *Proc. POPL 2016*.
- [17] The Coq Development Team. *The Coq Proof Assistant Reference Manual*. Version 8.19, 2024.